# <!-- SYNC IMPACT REPORT

Version Change : 1.3.0 → 1.4.0 Bump Type : MINOR — added governance hardening principles (XVII–XIX), frontend stack standardization, and new plan/task quality gates.

Modified Principles: None (existing principles preserved)

Added Principles:

- Principle XVII : Centralized Role-Based Authorization (RBAC)
- Principle XVIII: Boundary Sanitization and Validation Gate
- Principle XIX : Agent Self-Verification Mandate

Added Sections: None Removed Sections: None

Templates Requiring Updates: ☑ .specify/templates/plan-template.md — added gates for RBAC and Design System compliance ☑ .specify/templates/tasks-template.md — added explicit agent self-verification and RBAC/validation-gate checklist requirements ☑ .specify/templates/spec-template.md — no changes required ☑ .specify/templates/commands/*.md — no files found ☑ README.md / docs/quickstart.md — no files found

Deferred TODOs: None -->

# Clínica Digital Constitution

## Core Principles

### 0. Multi-Tenant Data Segregation (NON-NEGOTIABLE)

All tenant data MUST be completely isolated at every layer of the application stack. No data belonging to one tenant may ever be visible, accessible, or inferable by another tenant under any circumstance.

- **Tenant context** MUST be established at the request boundary (HTTP header or JWT claim) and propagated via Spring's request scope or a reactive `Context` through every application layer.
- **PostgreSQL Row-Level Security (RLS)** MUST be active on every table that contains tenant-scoped data. Migrations that introduce new tenant-scoped tables without a corresponding RLS policy MUST be rejected.
- Constructing, caching, or executing any query that crosses tenant boundaries is PROHIBITED. Super-admin cross-tenant aggregations require an explicit, separately secured service context and MUST generate an audit log entry for every access.
- Every tenant-scoped table MUST include a `tenant_id` column constrained by a foreign key to the tenants table; this column MUST be covered by the RLS policy.
- Integration test suites MUST include explicit cross-tenant isolation assertions for every resource type exposed by the application.

**Rationale**: Brazilian LGPD, CFM Resolution 1.821/2007, and enterprise SaaS security best practices require that a misconfiguration or breach isolated to one tenant cannot expose another tenant's clinical records.

## I. Test-First Development (NON-NEGOTIABLE)

All production code MUST be preceded by a failing automated test. The Red-Green- Refactor cycle is the only accepted development loop.

- Tests MUST be written and committed as failing before any implementation code is added. PRs that add implementation code without a prior-committed, failing test MUST be rejected.
- **Unit tests** MUST cover all domain and service-layer business logic.
- **Integration tests** MUST use Testcontainers backed by a real PostgreSQL instance; in-memory database substitutes are PROHIBITED for validating RLS policies, triggers, or pgcrypto-dependent behaviour.
- **Contract tests** MUST exist for every public module API surface and every FHIR resource exchanged with the RNDS.
- Coverage gates: ≥ 80% line coverage on domain/service layers; ≥ 90% on security-critical and multi-tenant isolation paths.

**Rationale**: In a regulated healthcare context, untested code is a patient safety and compliance risk. Test-First enforces clear, agreed requirements before any implementation begins.

## II. CLI Interface Mandate (NON-NEGOTIABLE)

Every library or module MUST expose its primary capability through a CLI interface in addition to any API or UI adapter.

- CLI commands MUST be deterministic, scriptable, and suitable for CI execution.
- Input and output contracts MUST support machine-readable JSON mode for automated validation and observability workflows.
- CLI entrypoints MUST be versioned with the module and covered by contract tests.
- A feature is not considered complete unless its core flow is executable through CLI without dependency on UI layers.

**Rationale**: CLI-first execution provides interface-agnostic testability, operability, and observability for regulated healthcare systems.

## III. Library-First Architecture (NON-NEGOTIABLE)

Every feature MUST be designed and proven as a standalone, reusable component before integration into the host application.

- A feature MUST be implemented as a separate Maven module (or a clearly bounded package with a stable public API) that can be built, tested, and published independently.
- Libraries MUST be self-contained: no circular dependencies; all external dependencies declared explicitly in the library's own `pom.xml`.
- All inter-module communication MUST be through declared, validated contracts: Java interfaces, immutable DTO records, or OpenAPI/AsyncAPI specifications. Direct access to another module's internal classes is PROHIBITED.
- Public library APIs MUST be versioned semantically; breaking changes require a MAJOR version bump and a documented migration path.
- A library is considered "ready for integration" only after: (a) contract tests pass, (b) a standalone quickstart document exists, and (c) a constitution review confirms adherence to all principles.

**Rationale**: Library-First prevents coupling in a complex healthcare domain, enables independent team delivery, and allows safe replacement or reuse of components across multiple clinic products.

### IV. Specification as Source of Truth (NON-NEGOTIABLE)

The feature specification (`spec.md`) is the primary artifact. Code is only a realization of the approved specification.

- No implementation MAY diverge from `spec.md`.
- Any required behavior change MUST first update `spec.md` and related design artifacts before code changes are merged.
- Every technical decision in `plan.md` MUST trace to at least one requirement ID in `spec.md`.
- Pull requests MUST include explicit traceability evidence between changed code, plan decisions, and specification requirements.

**Rationale**: In health software, undocumented behavior is operational and regulatory risk; the specification must govern implementation.

### V. Uncertainty Management (NON-NEGOTIABLE)

Ambiguity MUST be surfaced explicitly. Agents and developers MUST NOT invent clinical rules, compliance interpretations, or business behavior without confirmation.

- Any unresolved point in technical planning MUST be marked as `[NEEDS CLARIFICATION]`.
- Plans containing unresolved ambiguity MUST not proceed to implementation until clarifications are recorded or explicitly accepted by product/compliance owners.
- Clinical assumptions (care protocols, consent rules, RNDS mappings) are forbidden unless sourced from approved requirements.

**Rationale**: Hidden assumptions create patient safety risks and legal exposure.

### VI. Data-at-Rest Security, Error Standardization, and Audit Immutability (NON-NEGOTIABLE)

All sensitive health and personal data stored by the system MUST be protected using enterprise-grade security controls aligned with healthcare standards.

- Column-level encryption MUST be applied to fields classified as PHI (Protected Health Information) and sensitive PII (e.g., CPF, CRM, medical record numbers, diagnoses, prescriptions). AES-256-GCM or stronger is REQUIRED (via pgcrypto or application-layer encryption).
- Encryption keys MUST be managed externally using a dedicated secrets manager (HashiCorp Vault, AWS KMS, or equivalent). Hardcoded keys are PROHIBITED.
- Key rotation policies MUST be documented, automated where feasible, and validated at least annually.
- Database backups MUST be encrypted using the same standard or equivalent.
- Access to raw decrypted data MUST be restricted to the minimum necessary roles and MUST generate an immutable audit log entry.
- All API errors MUST conform to FHIR `OperationOutcome` with predictable severity, code, and diagnostics fields.
- Audit log tables are append-only; no service account may hold UPDATE or DELETE privileges on audit tables under any environment.

**Rationale**: LGPD Art. 46, CFM Resolution 1.821/2007, HL7 FHIR Security, and IHE ATNA mandate encryption and end-to-end audit trails for clinical data. Enterprise health systems are a high-value target; defence in depth is required.

## VII. Regulatory Compliance — RNDS, LGPD, HIPAA (NON-NEGOTIABLE)

The system MUST comply with Brazilian health data regulations and RNDS integration standards. HIPAA alignment is REQUIRED for any international or cross-border context.

- **RNDS/FHIR**: All data exchanged with the Rede Nacional de Dados em Saúde MUST conform to FHIR R4 resources using the official Brazilian RNDS profiles (StructureDefinitions published at simplifier.net/rnds). Profiles MUST be applied and validated (via HAPI FHIR validator) before any submission.
- **LGPD Consent**: Consent for data processing MUST be explicit, granular (per purpose), and revocable. Consent state MUST be persisted per patient per processing purpose with a complete, immutable audit trail.
- **Data Subject Rights**: The system MUST implement workflows for: right of access, right to correction, right to deletion (where legally permissible for health records), and right to data portability (FHIR Bundle export).
- **ROPA (Register of Processing Activities)**: A programmatic register of data processing activities MUST be maintained and be exportable on demand.
- **HIPAA Alignment**: Administrative, physical, and technical safeguards from the HIPAA Security Rule MUST be documented and mapped to implemented controls.
- Any change touching FHIR resource mappings or consent workflows MUST include a designated compliance review before merge is permitted.

**Rationale**: LGPD non-compliance carries fines up to 2 % of Brazilian revenue (capped at R$ 50 M per infraction). RNDS integration is a legal requirement for health information exchange in Brazil. HIPAA alignment future-proofs the product.

## VIII. Simplicity and Anti-Abstraction (NON-NEGOTIABLE)

The default architectural posture is simplicity. Premature abstraction is forbidden.

- New code MUST start with the simplest design that satisfies the current spec.
- Abstractions (extra layers, generic frameworks, inheritance hierarchies, indirection) are allowed only when a concrete design failure has been observed or a technical requirement is documented.
- Every approved complexity increase MUST be recorded in `Complexity Tracking` with: violation, need, rejected simpler alternative, and link to supporting evidence.

**Rationale**: Unnecessary abstraction reduces safety, debuggability, and delivery speed in high-regulation domains.

## IX. Drift Governance and Synchronization (NON-NEGOTIABLE)

The implementation state MUST remain a faithful reflection of specification and task artifacts at all times.

- CI/CD pipelines MUST execute drift-verification tooling (for example, `spec-kit-verify-tasks`) on every pull request.

- Any discrepancy between `tasks.md` completion state and the actual implementation evidence is a blocking failure and MUST prevent merge.
- "Phantom completion" (task marked done without corresponding code/tests/evidence) is prohibited and MUST be treated as a governance violation.
- Drift between `spec.md`, `plan.md`, `tasks.md`, and code MUST be resolved before approval; exceptions require explicit architect and compliance sign-off.

**Rationale**: Drift in regulated health platforms creates false assurance, weakens auditability, and increases patient and legal risk.

### X. Multi-Tenant Performance Isolation (NON-NEGOTIABLE)

Tenant isolation MUST include not only data boundaries but also performance and resource fairness.

- Every API and CLI adapter MUST enforce rate limiting and quota controls keyed by `tenant_id`.
- Resource ceilings (requests per minute, concurrency, queue depth, and batch size) MUST be configurable per tenant tier.
- A tenant exceeding quota MUST receive a deterministic, standards-compliant error response using FHIR `OperationOutcome`.
- Load and resilience tests MUST validate that one tenant cannot degrade latency or throughput guarantees for other tenants.

**Rationale**: Data segregation without performance isolation still allows noisy- neighbor outages and SLA breaches across clinical tenants.

### XI. Observability as a Design Contract (NON-NEGOTIABLE)

Observability is a design requirement, not a post-implementation add-on.

- Every business operation MUST define expected traces, metrics, and structured logs during planning.
- `trace_id` propagation is mandatory across synchronous calls and asynchronous flows (Kafka/RabbitMQ producers, consumers, retries, and dead-letter handlers).
- Structured JSON logs MUST include `tenant_id`, correlation identifiers, operation name, and outcome status.
- New features are incomplete without monitoring acceptance criteria and runtime dashboard/alert mappings.

**Rationale**: Reliable clinical systems require end-to-end diagnosability for safety, incident response, and compliance evidence.

### XII. Agent Context and Skills Governance (NON-NEGOTIABLE)

AI agents operating in this repository MUST follow project-specific instructions and must not apply conflicting context from unrelated sessions.

- Agents MUST load specialized instructions from `.github/skills` when available and relevant to the task domain.
- Agents MUST treat the current `spec.md` and this constitution as authoritative over any prior-session memory or defaults.
- Reusing stale assumptions that conflict with current constitutional or spec rules is prohibited.

- Any unresolved instruction conflict MUST be marked `[NEEDS CLARIFICATION]` and escalated before code generation.

**Rationale**: Deterministic agent behavior is required to keep governance, compliance, and architecture consistent across iterations.

### XIII. Architecture Exploration for High-Performance/Critical Modules (NON-NEGOTIABLE)

For modules classified as High Performance or Mission Critical, architecture options MUST be explored before implementation commitment.

- `plan.md` MUST evaluate at least two distinct implementation approaches (spikes) for each High Performance or Mission Critical module.
- Each spike MUST include measurable trade-offs: latency, throughput, operational risk, complexity, and tenant-isolation impact.
- Implementation tasks MUST not be finalized before spike outcomes and selection rationale are documented and approved.

**Rationale**: Premature convergence in critical healthcare modules increases outage risk and can compromise clinical continuity.

### XIV. Immutable Event Contracts (NON-NEGOTIABLE)

Event schemas used in Kafka/RabbitMQ MUST follow strict semantic versioning and consumer-safe evolution rules.

- Message contract changes MUST be versioned semantically and reviewed through a mandatory `Schema Compatibility Check`.
- Producers that introduce backward-incompatible payload changes MUST NOT merge without an approved consumer migration strategy.
- Compatibility evidence MUST be attached to PRs (schema diff + validation result).

**Rationale**: Event contract drift creates hidden breakages and delayed failures in clinical workflows that depend on asynchronous processing.

### XV. Graceful Degradation and Clinical Safety (NON-NEGOTIABLE)

External dependency failures MUST not halt critical clinical operations.

- Integrations with RNDS, KMS, Vault, and other external systems MUST implement circuit breaker patterns and bounded retry policies.
- Features relying on external integrations MUST implement a `Fallback Strategy` and temporary persistence using Outbox Pattern where applicable.
- Failures and degradation paths MUST preserve auditability and patient-safety constraints.

**Rationale**: Clinical continuity requires resilient behavior during dependency instability and partial outages.

### XVI. Supply Chain Security (NON-NEGOTIABLE)

Dependency transparency and provenance are mandatory for all released libraries.

- CI/CD MUST generate a Software Bill of Materials (SBOM) for every library release.
- SBOM artifacts MUST be stored with release evidence and remain traceable to commit, build pipeline, and package version.
- Releases without SBOM generation MUST be blocked.

**Rationale**: Healthcare-grade security requires full dependency visibility to support vulnerability response and compliance audits.

### XVII. Centralized Role-Based Authorization (RBAC) (NON-NEGOTIABLE)

Every exposed capability MUST enforce explicit clinical permission mapping in the service layer, independent of storage-level protections.

- RBAC MUST be centralized and policy-driven, with role-to-permission mappings versioned as code.
- No endpoint, CLI command, or asynchronous handler may execute clinical actions without a service-layer authorization check.
- Database controls (RLS/GRANTs) are defense-in-depth and MUST NOT be the primary authorization mechanism.
- Permission decisions MUST be auditable with role, permission key, principal, and tenant context.

**Rationale**: Clinical access boundaries depend on care roles and responsibilities, not only data partitioning; bypassing service-layer authorization is unacceptable.

### XVIII. Boundary Sanitization and Validation Gate (NON-NEGOTIABLE)

All incoming clinical data MUST pass sanitization and schema validation before persistence or downstream publication.

- Each module MUST implement a Sanitization and Validation Gate at boundaries (HTTP, CLI, messaging consumers, integration adapters).
- Payloads MUST be validated against approved FHIR profiles/schemas or strongly validated DTO contracts.
- Malformed, unsafe, or semantically invalid payloads MUST be rejected with FHIR `OperationOutcome` and MUST NOT be persisted.
- Validation and rejection events MUST be logged with `tenant_id`, `trace_id`, and validation error metadata.

**Rationale**: Boundary validation prevents propagation of corrupted clinical data and blocks malicious payloads early.

### XIX. Agent Self-Verification Mandate (NON-NEGOTIABLE)

Before marking any task as completed in `tasks.md`, the agent MUST perform an explicit consistency verification against this constitution.

- Agents MUST run a consistency checklist analysis (for example, `/speckit.checklist`) before changing task status to done.
- Any detected constitutional violation MUST block task completion until corrected or explicitly approved by architect/compliance.
- Checklist evidence MUST be linked in implementation notes or PR description.

**Rationale**: Prevents false completion and ensures automated work remains aligned with constitutional governance.

## Technology Stack

The following choices are canonical. Deviations MUST be ratified as a constitutional amendment.

| Concern | Choice |
|---|---|
| Language / Runtime | Java 21+ |
| Application Framework | Spring Boot 3.x |
| Security Framework | Spring Security + OAuth2/OIDC (Keycloak or equivalent) |
| Persistence | PostgreSQL 15+ with Row-Level Security and pgcrypto |
| FHIR Library | HAPI FHIR 7.x (client + server + validator) |
| Build / Modules | Maven multi-module; one Maven module per library |
| Testing | JUnit 5, Mockito, Testcontainers (PostgreSQL), RestAssured, Spring Boot Test |
| Observability | Spring Actuator, Micrometer, OpenTelemetry, Logback (JSON output) |
| Secrets Management | HashiCorp Vault or cloud KMS |
| Async / Events | Apache Kafka or RabbitMQ with schema registry |
| Frontend Framework | React 19+ with TypeScript |
| Styling | Tailwind CSS |
| Component Library | Material UI (MUI) 7 |

All UI development MUST be component-based and MUST conform to the project Design System definitions.

## Development Workflow

- **Feature branches**: All work MUST occur on feature branches; `main` is protected and requires passing CI and an approved PR.
- **Constitution Check** (in `plan.md`): MUST pass before Phase 0 research and MUST be re-verified after Phase 1 design:
    1. Multi-tenant strategy documented for every new entity (tenant_id column + RLS policy identified).
    2. Test plan present with contract, integration, and unit test categories.
    3. CLI contract identified for each new library/module, including JSON output mode.
    4. Library boundary defined with public API contract identified and named.
    5. `plan.md` decisions trace to `spec.md` requirements.
    6. Any ambiguity is marked `[NEEDS CLARIFICATION]` and resolved before coding.
    7. Security classification assigned to all new data fields (PHI / PII / non-sensitive).
    8. API error strategy mapped to FHIR `OperationOutcome`.
    9. RNDS/LGPD impact assessed; compliance review flagged if resources or consent workflows are affected.

10. Complexity additions are documented in `Complexity Tracking` with evidence.
11. Drift-verification tooling is planned in CI/CD (for example, `spec-kit-verify-tasks`) and configured to block merge on divergence.
12. Monitoring plan maps tracing and metrics for the new feature, including `trace_id` propagation and tenant-context logging.
13. For any new endpoint/command, tenant-based rate limiting and quota strategy is specified.
14. For High Performance/Mission Critical modules, at least two architecture spikes are evaluated before implementation tasks are approved.
15. Event-driven changes include mandatory `Schema Compatibility Check` results and migration strategy for breaking schema changes.
16. External integrations define `Fallback Strategy` and resilience controls (circuit breaker, retries, outbox when applicable).
17. Release plan includes SBOM generation and retention evidence in CI/CD. 18. RBAC permissions are mapped explicitly for each service operation and linked to clinical role requirements. 19. UI changes include Design System conformance checks (component usage, accessibility, and token adherence). 20. Boundary Sanitization and Validation Gate is defined for all new data ingress paths (HTTP/CLI/events/integrations).

- **Code Review**: Reviewers MUST explicitly verify: multi-tenant isolation, test coverage above thresholds, CLI parity, traceability to spec, encryption of new PHI/PII fields, OperationOutcome conformance, observability contract evidence, quota/rate-limit enforcement, RBAC mapping completeness, Design System adherence, and library contract compliance.
- **CI Gates**: Build MUST pass all tests (including Testcontainers integration tests), coverage thresholds, SpotBugs, SonarQube quality gate, and OWASP Dependency-Check before merge is permitted. CI MUST also pass drift verification between `spec.md`, `plan.md`, `tasks.md`, and implementation evidence. Release pipelines MUST generate and publish SBOM artifacts.
- **Migrations**: Every database migration MUST include RLS policy updates for new tenant-scoped tables. A rollback script is REQUIRED for every migration.

## Governance

This constitution supersedes all other development guidelines within this project.

- **Amendments** require: (a) a written proposal documenting the change and rationale, (b) consensus from the core team or designated architect approval, (c) a semantic version bump, and (d) propagation of changes to all dependent templates and artifacts.
- **Version Semantics** (normative):
  - MAJOR: Removal, weakening, or incompatible redefinition of a principle or mandatory gate.
  - MINOR: Addition of principle(s), mandatory section(s), or materially expanded governance and quality requirements.
  - PATCH: Clarifications, wording improvements, typo fixes, and non-semantic edits that do not alter obligations.
- **Governance Versioning Matrix**:

| Change Type | Required Bump | Example |
|---|---|---|
| Remove NON-NEGOTIABLE principle | MAJOR | Delete rate-limiting obligation |
| Relax CI blocking rule | MAJOR | Drift check becomes warning-only |

| Change Type | Required Bump | Example |
|---|---|---|
| Add a new constitutional article | MINOR | Add Observability article |
| Add new mandatory plan/task checkpoint | MINOR | Add quota verification gate |
| Clarify wording only | PATCH | Improve rationale text |

- **Compliance Review**: A constitution compliance check MUST occur at feature planning (plan.md gate) and at each PR review.
- All agents and tooling operating on this repository MUST consult `.specify/memory/constitution.md` before generating plans, specs, or tasks.

**Version**: 1.4.0 | **Ratified**: 2026-04-05 | **Last Amended**: 2026-04-05